Gaussian Representation of Bounding Boxes in ReDiffDet

A Step-by-Step Mathematical Explanation

Contents

1 The central idea **2**

2 What exactly is a bounding box? **2**

3 Representing the box by a two-dimensional Gaussian **3**
- 3.1 A Gaussian as a cloud of points . . . 3
- 3.2 The mean represents the box centre . . . 3
- 3.3 The covariance represents size and orientation . . . 3
- 3.4 Expanding the covariance formula . . . 4
- 3.5 A numerical illustration . . . 4

4 Sampling points from the Gaussian **5**
- 4.1 A small illustrative sample . . . 5
- 4.2 Why sample points? . . . 5

5 Coordinate transformations and signal scaling **6**
- 5.1 Rotating points . . . 6
- 5.2 Scaling the point cloud . . . 6

6 Adding Gaussian noise: the forward diffusion step **6**
- 6.1 The basic idea . . . 6
- 6.2 A scalar numerical example . . . 7
- 6.3 Vector form for sampled points . . . 7

7 Estimating a Gaussian from a set of points **7**
- 7.1 Estimating the mean . . . 8
- 7.2 Estimating the covariance . . . 8
- 7.3 Why does the MLE use $1/k$? . . . 8
- 7.4 A crucial distinction: fitting a Gaussian is not denoising . . . 9

8 Converting a Gaussian back into an oriented box **9**
- 8.1 Illustrative reconstruction . . . 10

9 Putting the complete pipeline together **10**

10 How is this different from DiffusionDet? **10**
- 10.1 DiffusionDet: diffusion over box coordinates . . . 10
- 10.2 ReDiffDet: Gaussian representation of oriented geometry . . . 11

11 Common confusions to avoid **11**

12 Compact equation summary **11**

13 Final intuition **12**

1 The central idea

An object detector ultimately needs to predict a bounding box around each object in an image. In an axis-aligned detector, a box can be represented by four numbers,

$$B = (c_x, c_y, w, h),$$

where (c_x, c_y) is the centre and w, h are the width and height. For an oriented bounding box (OBB), we additionally need an angle:

$$B = (c_x, c_y, w, h, \theta).$$

The angle specifies the orientation of the rectangle.

ReDiffDet uses a Gaussian representation to encode the geometry of an oriented box. The Gaussian is not saying that the object itself has a Gaussian shape. Rather, it is a convenient mathematical representation whose mean describes the box centre and whose covariance describes its spatial spread and orientation.

The conceptual pipeline discussed in these notes is

box $\longrightarrow$ Gaussian $\longrightarrow$ sampled points $\longrightarrow$ noisy points $\longrightarrow$ estimated/predicted Gaussian $\longrightarrow$ box.

Each stage is explained below. The precise implementation details, parameter conventions, and loss functions should always be checked against the particular ReDiffDet paper or code version being used. Where a formula is illustrative rather than a verbatim implementation equation, it is identified as such.

2 What exactly is a bounding box?

A bounding box is a rectangular region that encloses an object. Its four edges form the boundary, and the area inside those edges is the region. It is not generally the exact silhouette of the object: some background pixels may lie inside the rectangle.

For an oriented rectangle, write

$$B = (c_x, c_y, w, h, \theta).$$

Here:

- c_x, c_y: coordinates of the rectangle centre in the image;
- w: side length along the box's width direction;
- h: side length along the box's height direction;
- θ: rotation angle relative to the image coordinate axes.

For example,

$$B = (100, 80, 20, 10, 30^\circ)$$

describes a rectangle centred at $(100, 80)$, with width 20, height 10, and orientation 30°.

The box is therefore a geometric object described by five parameters. ReDiffDet's Gaussian representation provides another way of encoding these same geometric properties.

3 Representing the box by a two-dimensional Gaussian

3.1 A Gaussian as a cloud of points

A two-dimensional Gaussian distribution is written as

$$P \sim \mathcal{N}(\mu, \Sigma),$$

where

$$\mu = \begin{bmatrix} \mu_x \\ \mu_y \end{bmatrix} \qquad \text{and} \qquad \Sigma = \begin{bmatrix} \sigma_{xx} & \sigma_{xy} \\ \sigma_{xy} & \sigma_{yy} \end{bmatrix}.$$

The mean μ is the centre of the point cloud. The covariance matrix Σ describes how the points are spread around that centre:

- larger variance means a wider spread in the corresponding direction;
- smaller variance means a tighter spread;
- unequal variances produce an elongated cloud;
- nonzero off-diagonal covariance can indicate that the cloud is tilted relative to the coordinate axes.

Thus, the mean and covariance can encode the centre, scale, and orientation of a rectangular box.

3.2 The mean represents the box centre

For a box with centre (c_x, c_y), choose

$$\boxed{\mu = \begin{bmatrix} c_x \\ c_y \end{bmatrix}.}$$

For the example box,

$$\mu = \begin{bmatrix} 100 \\ 80 \end{bmatrix}.$$

This places the centre of the Gaussian point cloud at the centre of the rectangle.

3.3 The covariance represents size and orientation

First consider an axis-aligned box, whose width and height directions coincide with the image axes. A simple covariance construction is

$$\Sigma_0 = \begin{bmatrix} \sigma_w^2 & 0 \\ 0 & \sigma_h^2 \end{bmatrix}.$$

The standard deviations σ_w and σ_h control the spread along the horizontal and vertical directions.

To represent an oriented box, rotate the covariance. The two-dimensional rotation matrix is

$$R(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}.$$

The rotated covariance is

$$\boxed{\Sigma = R(\theta)\Sigma_0 R(\theta)^{\mathsf{T}}.}$$

The rightmost transpose is important: it rotates the covariance consistently in both coordinate directions.

A scale convention used in the discussion of ReDiffDet is

$$\boxed{\Sigma = R(\theta) \begin{bmatrix} (w/m)^2 & 0 \\ 0 & (h/m)^2 \end{bmatrix} R(\theta)^\mathsf{T},}$$

where m is a scale factor. This convention makes the standard deviations proportional to the box dimensions. Increasing m makes the Gaussian tighter; decreasing m makes it wider. The exact value or role of m is implementation-dependent.

3.4 Expanding the covariance formula

Let

$$a = (w/m)^2, \qquad b = (h/m)^2, \qquad c = \cos\theta, \quad s = \sin\theta.$$

Then

$$R = \begin{bmatrix} c & -s \\ s & c \end{bmatrix}, \qquad R^\mathsf{T} = \begin{bmatrix} c & s \\ -s & c \end{bmatrix}.$$

First multiply the first two matrices:

$$R \begin{bmatrix} a & 0 \\ 0 & b \end{bmatrix} = \begin{bmatrix} ac & -bs \\ as & bc \end{bmatrix}.$$

Now multiply by R^T:

$$\Sigma = \begin{bmatrix} ac & -bs \\ as & bc \end{bmatrix} \begin{bmatrix} c & s \\ -s & c \end{bmatrix}.$$

Consequently,

$$\boxed{\Sigma = \begin{bmatrix} ac^2 + bs^2 & (a-b)cs \\ (a-b)cs & as^2 + bc^2 \end{bmatrix}.}$$

This expanded form shows how the angle affects the entries of the covariance matrix. When $\theta = 0$, $c = 1$ and $s = 0$, so

$$\Sigma = \begin{bmatrix} a & 0 \\ 0 & b \end{bmatrix}.$$

When the box is rotated, the off-diagonal terms are generally nonzero.

3.5 A numerical illustration

Take

$$B = (100, 80, 20, 10, 30^\circ), \qquad m = 2.$$

Then

$$a = (20/2)^2 = 100, \qquad b = (10/2)^2 = 25.$$

Using $\cos 30^\circ = \sqrt{3}/2$ and $\sin 30^\circ = 1/2$,

$$\Sigma = \begin{bmatrix} 100(3/4) + 25(1/4) & (100-25)(\sqrt{3}/4) \\ (100-25)(\sqrt{3}/4) & 100(1/4) + 25(3/4) \end{bmatrix}.$$

Therefore,

$$\Sigma \approx \begin{bmatrix} 81.25 & 32.48 \\ 32.48 & 43.75 \end{bmatrix}.$$

The Gaussian representation of this box is

$$P \sim \mathcal{N}\left(\begin{bmatrix}100\\80\end{bmatrix}, \begin{bmatrix}81.25 & 32.48\\32.48 & 43.75\end{bmatrix}\right).$$

The mean gives the centre. The covariance describes an elongated point cloud whose principal directions are aligned with the oriented box.

4 Sampling points from the Gaussian

Once the Gaussian parameters are known, we can sample points from it. If

$$P \sim \mathcal{N}(\mu, \Sigma),$$

then one standard way to generate a sample is

$$\boxed{P = \mu + Lu,}$$

where $u \sim \mathcal{N}(0, I)$ and L is a matrix satisfying

$$LL^{\mathsf{T}} = \Sigma.$$

For example, L can be obtained by a Cholesky factorization when Σ is positive definite.

For a two-dimensional point,

$$u = \begin{bmatrix}u_x\\u_y\end{bmatrix}, \qquad u_x, u_y \sim \mathcal{N}(0, 1).$$

The random vector u is first transformed by L, which gives it the required spread and orientation, and then shifted by μ to place the cloud at the box centre.

4.1 A small illustrative sample

Suppose the standard-normal draws are

$$u_1 = \begin{bmatrix}0.2\\-0.4\end{bmatrix}, \quad u_2 = \begin{bmatrix}-0.7\\0.1\end{bmatrix}, \quad u_3 = \begin{bmatrix}1.0\\0.5\end{bmatrix}.$$

For each draw, form

$$P_i = \mu + Lu_i.$$

The resulting points are random locations around the centre, with the covariance-controlled spread. The numerical coordinates depend on the chosen factor L and the random draws.

A configuration may use k sampled points (the discussion here used $k = 8$). The points are not required to lie on the rectangle boundary or even inside the rectangle. They are samples from the Gaussian distribution used to represent the box.

4.2 Why sample points?

The point set gives a finite geometric representation of the Gaussian. It can be transformed and corrupted by a diffusion process. After corruption, its sample statistics can be used to estimate a Gaussian again. This makes the Gaussian representation usable in a point-based diffusion pipeline.

5 Coordinate transformations and signal scaling

5.1 Rotating points

For a point expressed relative to the centre,

$$q = \begin{bmatrix} x \\ y \end{bmatrix},$$

a rotation by angle θ is

$$q' = R(\theta)q = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}.$$

In scalar form,

$$x' = x\cos\theta - y\sin\theta, \qquad y' = x\sin\theta + y\cos\theta.$$

For example, rotating $(1, 0)$ by 90° gives $(0, 1)$.

If the point is an absolute image coordinate rather than a centre-relative coordinate, rotate around the box centre:

$$P' = \mu + R(\theta)(P - \mu).$$

This distinction matters: rotating an absolute point around the image origin would also move the box centre.

5.2 Scaling the point cloud

Scaling changes the spread of the point cloud. If a centre-relative point q is scaled by a scalar γ, then

$$q' = \gamma q.$$

Its covariance changes according to

$$\mathrm{Cov}(q') = \gamma^2 \,\mathrm{Cov}(q).$$

Thus, multiplying coordinates by a scale factor multiplies the covariance by the square of that factor.

In the covariance parameterization

$$\Sigma = R(\theta) \begin{bmatrix} (w/m)^2 & 0 \\ 0 & (h/m)^2 \end{bmatrix} R(\theta)^\top,$$

the factor m controls the point-cloud scale inversely: larger m gives smaller variances. This is the geometric meaning of the signal scaling in the representation.

6 Adding Gaussian noise: the forward diffusion step

6.1 The basic idea

Diffusion deliberately corrupts clean data by adding random Gaussian noise. In a simple scalar example,

$$\text{noisy value} = \text{scaled clean value} + \text{scaled random noise}.$$

A common variance-preserving diffusion parameterization is

$$\boxed{z_t = \sqrt{\bar{\alpha}_t}\, z_0 + \sqrt{1 - \bar{\alpha}_t}\, \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I).}$$

Here:

- z_0 is the clean geometric representation;
- z_t is the representation at diffusion timestep t;
- ε is standard Gaussian noise;
- $\bar{\alpha}_t$ controls how much of the clean signal remains.

This is a standard diffusion-style equation. It is included to explain the forward-noising idea; the exact ReDiffDet implementation may use its own notation, scaling, or parameterization.

6.2 A scalar numerical example

Consider one coordinate with

$$z_0 = 10, \qquad \varepsilon = 2.$$

If $\bar{\alpha}_t = 0.9$, then

$$z_t = \sqrt{0.9}(10) + \sqrt{0.1}(2) \approx 9.4868 + 0.6325 \approx 10.1193.$$

The clean signal still contributes strongly.

If $\bar{\alpha}_t = 0.1$, then

$$z_t = \sqrt{0.1}(10) + \sqrt{0.9}(2) \approx 3.1623 + 1.8974 \approx 5.0597.$$

The clean signal contributes less, while the noise contributes more.

The same idea applies to the geometric representation. Instead of corrupting one scalar, the process corrupts the point coordinates or the chosen representation of the sampled geometry.

6.3 Vector form for sampled points

If each point is two-dimensional, write

$$P_i^{(0)} = \begin{bmatrix} x_i^{(0)} \\ y_i^{(0)} \end{bmatrix}.$$

A coordinate-wise isotropic noising illustration is

$$P_i^{(t)} = \sqrt{\bar{\alpha}_t}\, P_i^{(0)} + \sqrt{1-\bar{\alpha}_t}\, \varepsilon_i, \qquad \varepsilon_i \sim \mathcal{N}(0, I_2).$$

For k points, this operation is applied to each point. The resulting set becomes progressively less informative about the original point configuration as the noise level increases.

The image features condition the learned reverse process: the network uses the image to infer which noisy geometric configurations are compatible with objects in that image.

7 Estimating a Gaussian from a set of points

After obtaining a set of points, we can summarize their empirical distribution by estimating its mean and covariance. This is the connection to the maximum-likelihood derivation discussed earlier.

Let the observed two-dimensional points be

$$P_1, P_2, \ldots, P_k \in \mathbb{R}^2.$$

7.1 Estimating the mean

The sample mean is

$$\boxed{\hat{\mu} = \frac{1}{k}\sum_{i=1}^{k} P_i.}$$

If

$$P_i = \begin{bmatrix} x_i \\ y_i \end{bmatrix},$$

then

$$\hat{\mu} = \begin{bmatrix} \frac{1}{k}\sum_i x_i \\ \frac{1}{k}\sum_i y_i \end{bmatrix}.$$

It is simply the average x-coordinate and the average y-coordinate. It gives the centre of the observed point cloud.

7.2 Estimating the covariance

The Gaussian maximum-likelihood covariance estimate is

$$\boxed{\hat{\Sigma}_{\text{MLE}} = \frac{1}{k}\sum_{i=1}^{k}(P_i - \hat{\mu})(P_i - \hat{\mu})^{\mathsf{T}}.}$$

To understand one term, write

$$P_i - \hat{\mu} = \begin{bmatrix} x_i - \hat{\mu}_x \\ y_i - \hat{\mu}_y \end{bmatrix}.$$

Its outer product is

$$(P_i - \hat{\mu})(P_i - \hat{\mu})^{\mathsf{T}} = \begin{bmatrix} (x_i - \hat{\mu}_x)^2 & (x_i - \hat{\mu}_x)(y_i - \hat{\mu}_y) \\ (x_i - \hat{\mu}_x)(y_i - \hat{\mu}_y) & (y_i - \hat{\mu}_y)^2 \end{bmatrix}.$$

Averaging these matrices gives

$$\hat{\Sigma}_{\text{MLE}} = \begin{bmatrix} \widehat{\text{Var}}(x) & \widehat{\text{Cov}}(x,y) \\ \widehat{\text{Cov}}(x,y) & \widehat{\text{Var}}(y) \end{bmatrix}.$$

The diagonal entries measure spread along the coordinate axes. The off-diagonal entries measure how the two coordinates vary together.

7.3 Why does the MLE use $1/k$?

For independent samples from $\mathcal{N}(\mu, \Sigma)$, maximizing the Gaussian likelihood with respect to μ gives $\hat{\mu} = k^{-1}\sum_i P_i$. Substituting this mean into the log-likelihood and differentiating with respect to Σ gives

$$-\frac{k}{2}\Sigma^{-1} + \frac{1}{2}\Sigma^{-1}S\Sigma^{-1} = 0, \qquad S = \sum_{i=1}^{k}(P_i - \hat{\mu})(P_i - \hat{\mu})^{\mathsf{T}}.$$

Multiplying on the left and right by Σ yields

$$-k\Sigma + S = 0,$$

and therefore

$$\hat{\Sigma}_{\text{MLE}} = \frac{S}{k}.$$

The unbiased sample covariance instead uses $1/(k-1)$. These are different estimators: the first is the Gaussian maximum-likelihood estimate, while the second corrects the finite-sample bias caused by estimating the mean from the same data.

7.4 A crucial distinction: fitting a Gaussian is not denoising

Computing $\hat{\mu}$ and $\hat{\Sigma}$ from a point set is a statistical summarization step. It tells us the empirical centre and spread of those points. It does not, by itself, recover the original clean points or remove diffusion noise.

The learned denoising network is a separate component. It receives noisy geometric information together with image features and learns to predict or reconstruct a cleaner representation. MLE can then be used as a way to summarize a point set, depending on the method's particular pipeline.

8 Converting a Gaussian back into an oriented box

Suppose a predicted Gaussian is

$$P \sim \mathcal{N}(\hat{\mu}, \hat{\Sigma}).$$

The centre of the predicted box is obtained from its mean:

$$\begin{bmatrix} \hat{c}_x \\ \hat{c}_y \end{bmatrix} = \hat{\mu}.$$

To understand the spread and orientation, use the eigendecomposition of the symmetric covariance matrix:

$$\hat{\Sigma} = Q\Lambda Q^{\mathsf{T}}, \qquad \Lambda = \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix}.$$

The eigenvectors (columns of Q) give the principal directions of the point cloud. The eigenvalues give the variances along those directions. The corresponding standard deviations are

$$\sqrt{\lambda_1}, \qquad \sqrt{\lambda_2}.$$

Thus:

- the mean gives the box centre;
- the principal eigenvector gives a principal orientation;
- the eigenvalues describe the spread along the two principal axes.

  If the covariance was constructed using

$$\Sigma = R(\theta) \begin{bmatrix} (w/m)^2 & 0 \\ 0 & (h/m)^2 \end{bmatrix} R(\theta)^{\mathsf{T}},$$

then, under the same convention and assuming the width axis corresponds to the first principal direction,

$$w = m\sqrt{\lambda_1}, \qquad h = m\sqrt{\lambda_2},$$

and θ is read from the corresponding eigenvector. The ordering of eigenvalues and axes must be handled consistently; otherwise width and height can be swapped or the angle can differ by $90°$.

For a two-dimensional eigenvector $q_1 = (q_{1x}, q_{1y})^{\mathsf{T}}$, its direction angle is

$$\theta = \operatorname{atan2}(q_{1y}, q_{1x}).$$

The resulting box is

$$\hat{B} = (\hat{c}_x, \hat{c}_y, \hat{w}, \hat{h}, \hat{\theta}).$$

8.1 Illustrative reconstruction

Suppose the predicted Gaussian has mean

$$\hat{\mu} = \begin{bmatrix} 100 \\ 80 \end{bmatrix},$$

and its principal directions and spreads correspond, under the chosen scale convention, to width 20, height 10, and angle 30°. Then the reconstructed oriented box is

$$\hat{B} = (100, 80, 20, 10, 30^\circ).$$

This example illustrates the mapping from Gaussian parameters back to box parameters. It is not a claim that every implementation uses precisely this numerical scale convention.

9 Putting the complete pipeline together

The stages can now be interpreted as follows.

1. **Start with the ground-truth box.** The annotation gives the centre, dimensions, and (for an OBB) orientation.
2. **Construct a Gaussian.** The mean is set to the box centre. The covariance is constructed from the dimensions and orientation, with a chosen scale convention.
3. **Sample points.** Draw k points from the Gaussian. These points form a finite representation of its geometry.
4. **Transform and scale.** Apply the coordinate transformation and scaling required by the representation. Rotations must be applied consistently to the point coordinates and covariance.
5. **Apply forward diffusion.** Add Gaussian noise according to the selected noise schedule. The noisy points become less directly informative about the original geometry.
6. **Predict or estimate a Gaussian.** The learned network performs the denoising prediction using image features. Sample mean and covariance formulas can summarize a point set, but computing those statistics alone is not the learned denoising step.
7. **Recover a bounding box.** Read the centre from the mean and infer the principal directions and spreads from the covariance. Convert them to width, height, and angle using the same parameterization used to construct the Gaussian.

10 How is this different from DiffusionDet?

The two methods should not be conflated.

10.1 DiffusionDet: diffusion over box coordinates

DiffusionDet represents candidate boxes using their box coordinates. In a simplified illustration, a box can be written as

$$b = (c_x, c_y, w, h).$$

The forward process corrupts these box-coordinate vectors with Gaussian noise according to a diffusion schedule. A denoising network uses image features to refine noisy candidate boxes toward object boxes.

The key idea is that DiffusionDet diffuses box-coordinate representations. It does not need to convert each box into a two-dimensional Gaussian point cloud as a prerequisite.

10.2 ReDiffDet: Gaussian representation of oriented geometry

In the Gaussian representation described here, an oriented box is encoded by

$$(\mu, \Sigma),$$

where μ represents its centre and Σ represents its dimensions and orientation. Points can be sampled from this Gaussian, transformed, and used in the diffusion-based geometric process. The recovered Gaussian can then be converted back into an oriented box.

Aspect	**DiffusionDet**	**ReDiffDet representation discussed here**
Geometric representation	Box-coordinate vector	Gaussian mean and covariance
Geometry encoded	Centre and dimensions	Centre, spread, and orientation
Noising concept	Noise applied to box coordinates	Noise applied to the selected Gaussian/point representation
Output	Refined bounding boxes	Recovered Gaussian converted to oriented boxes

The essential distinction is not that one method uses probability and the other does not: diffusion itself is probabilistic. The distinction is the geometric representation on which the diffusion process is defined.

11 Common confusions to avoid

1. **A box is a region, not a Gaussian.** The box is a rectangle enclosing the object. The Gaussian is an alternative mathematical encoding of its geometry.
2. **The Gaussian cloud is not the object's shape.** Its points are samples from a probability distribution constructed from the box parameters.
3. **Sampled points need not lie on the box boundary.** They are drawn from the Gaussian and can fall inside or outside the rectangle.
4. **The covariance is not the same thing as width and height.** It stores variances and correlations. Width and height are recovered through the chosen scale convention and principal directions.
5. **MLE is not the neural denoiser.** MLE estimates the mean and covariance of a supplied point set. A neural network learns the reverse/noise-prediction mapping.
6. **Do not mix parameterizations.** If the forward process uses scaled coordinates, the reconstruction must undo the same scaling. Likewise, the width/height ordering and angle convention must remain consistent.
7. **Do not assume the illustrative DDPM equation is the exact ReDiffDet equation.** The equation with $\bar{\alpha}_t$ explains a common Gaussian diffusion mechanism. For exact reproduction, use the precise schedule and parameterization from the paper and its implementation.

12 Compact equation summary

Oriented bounding box:

$$B = (c_x, c_y, w, h, \theta).$$

Gaussian representation:

$$P \sim \mathcal{N}(\mu, \Sigma), \qquad \mu = \begin{bmatrix} c_x \\ c_y \end{bmatrix}.$$

Rotation matrix:

$$R(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}.$$

Example covariance parameterization:

$$\Sigma = R(\theta) \begin{bmatrix} (w/m)^2 & 0 \\ 0 & (h/m)^2 \end{bmatrix} R(\theta)^{\mathsf{T}}.$$

Gaussian sampling:

$$P_i = \mu + Lu_i, \qquad u_i \sim \mathcal{N}(0, I), \qquad LL^{\mathsf{T}} = \Sigma.$$

Illustrative forward diffusion:

$$z_t = \sqrt{\bar{\alpha}_t}\, z_0 + \sqrt{1 - \bar{\alpha}_t}\, \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I).$$

Sample mean:

$$\hat{\mu} = \frac{1}{k} \sum_{i=1}^{k} P_i.$$

Gaussian MLE covariance:

$$\hat{\Sigma}_{\text{MLE}} = \frac{1}{k} \sum_{i=1}^{k} (P_i - \hat{\mu})(P_i - \hat{\mu})^{\mathsf{T}}.$$

Covariance eigendecomposition:

$$\hat{\Sigma} = Q \Lambda Q^{\mathsf{T}}.$$

Reconstruction under the stated scale convention:

$$\hat{c}_x, \hat{c}_y \leftarrow \hat{\mu}, \qquad \hat{w} = m\sqrt{\lambda_1}, \qquad \hat{h} = m\sqrt{\lambda_2},$$

with the angle obtained from the corresponding principal eigenvector, subject to consistent axis ordering.

13 Final intuition

Think of the bounding box as a rectangle described by its centre, size, and orientation. ReDiffDet's Gaussian representation replaces that direct geometric description with a point cloud: the mean places the cloud, and the covariance determines how it spreads and which direction it is tilted. Sampling turns the Gaussian into finitely many points. Diffusion corrupts the chosen geometric representation with noise, and a learned network uses image information to reverse that corruption. Finally, the mean and covariance of the recovered representation can be interpreted as the centre, dimensions, and orientation of an oriented bounding box.

The most important conceptual distinction is:

DiffusionDet: diffuse box coordinates	versus	ReDiffDet: use a Gaussian representation of oriented geo